

DEVOPS AUTOMATION FOR DATA ENGINEERING USING TERRAFORM ON AWS

Infrastructure as Code (IaC) + CI/CD for Modern Data Pipelines

1. PROJECT OVERVIEW

This project demonstrates **production-grade Infrastructure as Code (IaC)** for Data Engineering workloads on AWS using **Terraform** combined with a robust **GitHub Actions CI/CD pipeline**. It provisions a complete data ingestion foundation (S3 Data Lake + Glue Catalog + Scheduled Crawler) with full automation, security scanning, testing, and safe deployment practices.

Use Case: Temperature data analysis pipeline — Raw CSV data is generated, uploaded to S3, automatically cataloged by Glue Crawler, and made queryable via Athena. The entire infrastructure is managed declaratively through Terraform and deployed via CI/CD.

2. SYSTEM ARCHITECTURE

The architecture follows modern **DevOps + Data Engineering** best practices with full automation.

3. TECHNOLOGY STACK

Category	Technology	Purpose
Infrastructure as Code	Terraform v1.15.7	Declarative provisioning of AWS resources
Cloud Platform	Amazon Web Services (ap-south-1)	S3, Glue, IAM
Data Catalog	AWS Glue Data Catalog + Crawler	Automated metadata & schema discovery
CI/CD	GitHub Actions	Security scan, tests, Terraform pipeline
Testing	pytest + pytest-cov	Unit tests for data generation scripts
Security	Bandit + Safety	Static analysis & dependency vulnerability scanning
Query Engine	Amazon Athena	Serverless querying on Glue Catalog

4. PROJECT STRUCTURE

- Key Components:**
 - scripts/generate_sample_data.py — Synthetic temperature data generator (5 cities, realistic ranges, 365-day spread)
 - tests/test_generate_sample_data.py — Comprehensive pytest suite (7 tests covering file creation, content, edge cases)
 - terraform/main.tf — Core IaC (S3 bucket, Glue Database, Crawler, IAM Role, encryption, safe destruction)

- [.github/workflows/deploy.yml](#) — Full CI/CD pipeline (Security → Test → Validate → Plan → Deploy)

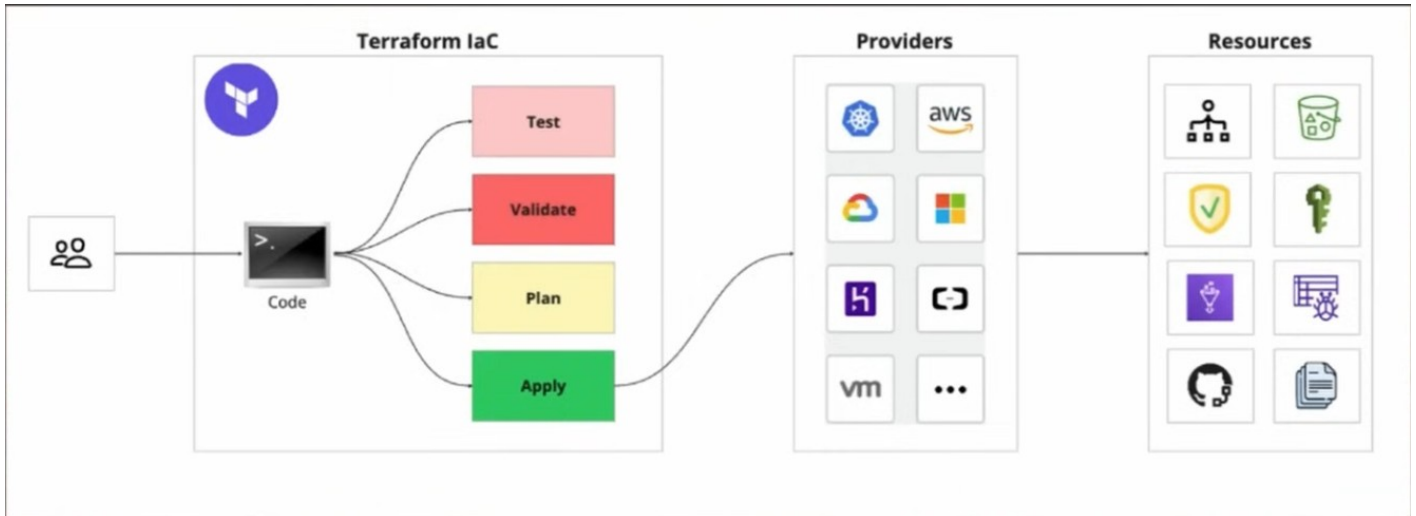


Figure 1: Project architecture

5. KEY IMPLEMENTATION DETAILS

5.1 Terraform Best Practices Implemented

```

1  data "aws_s3_bucket" "existing" {
2    count = var.use_existing_bucket ? 1 : 0
3    bucket = var.existing_bucket_name
4  }
5
6  locals {
7    bucket_name = var.use_existing_bucket ? var.existing_bucket_name : "${var.project_name}-hls-raw-data"
8  }
9
10 resource "null_resource" "empty_s3_bucket" {
11   triggers = {
12     bucket_name = local.bucket_name
13   }
14
15   provisioner "local-exec" {
16     command = "aws s3 rm s3://${local.bucket_name} --recursive"
17   }
18
19   depends_on = [aws_s3_bucket.raw_data]
20 }
21
22 resource "aws_s3_bucket" "raw_data" {
23   bucket = "aws-temperature-analysis-hls-raw-data"
24   force_destroy = true
25
26   tags = {
27     Name = "aws-temperature-analysis-hls-raw-data"
28     Environment = "dev"
29   }
30
31   lifecycle {
32     create_before_destroy = true
33   }
34 }
35
36 resource "aws_s3_bucket_versioning" "raw_data" {
37   bucket = aws_s3_bucket.raw_data.id
38   versioning_configuration {
39     status = "Disabled"
40   }
41 }
42
43 resource "aws_s3_bucket_server_side_encryption_configuration" "raw_data" {
44   bucket = aws_s3_bucket.raw_data.id
45
46   rule {
47     apply_server_side_encryption_by_default {
48       sse_algorithm = "AES256"
49     }
50   }
51 }

```

Figure 2: main.tf — S3 bucket with force_destroy, create_before_destroy lifecycle, AES-256 encryption, and null_resource for safe bucket emptying

- create_before_destroy lifecycle on S3, Glue resources, and IAM role to prevent downtime during updates
- null_resource with local-exec to recursively empty S3 bucket before destruction (critical for force_destroy)
- Server-side encryption (AES-256) enabled by default on S3 bucket
- Dedicated IAM role with least-privilege managed policies (AWSGlueServiceRole + AmazonS3FullAccess)

- Glue Crawler configured with TableLevelConfiguration = 3 and daily schedule (cron(0 1 * * ? *))

5.2 CI/CD Pipeline (GitHub Actions)

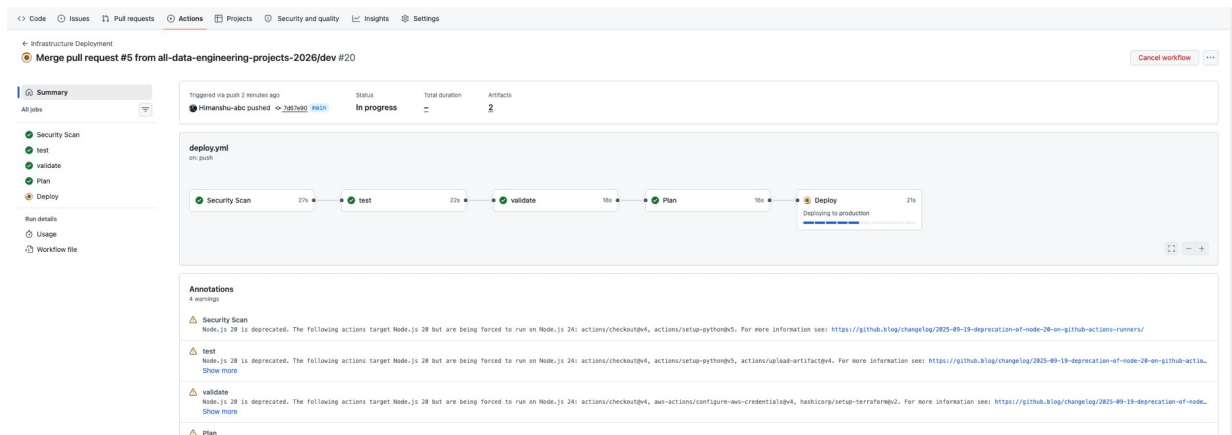


Figure 3: GitHub Actions workflow running for prod— Security Scan → Test → Validate → Plan → Deploy stages

- **Security Job:** Bandit static code analysis + Safety dependency scanning
- **Test Job:** pytest with coverage reporting (artifact uploaded)
- **Validate Job:** terraform init + terraform validate
- **Plan Job:** terraform plan (tfplan artifact uploaded for review)
- **Deploy Job:** Runs only on push to main with environment protection

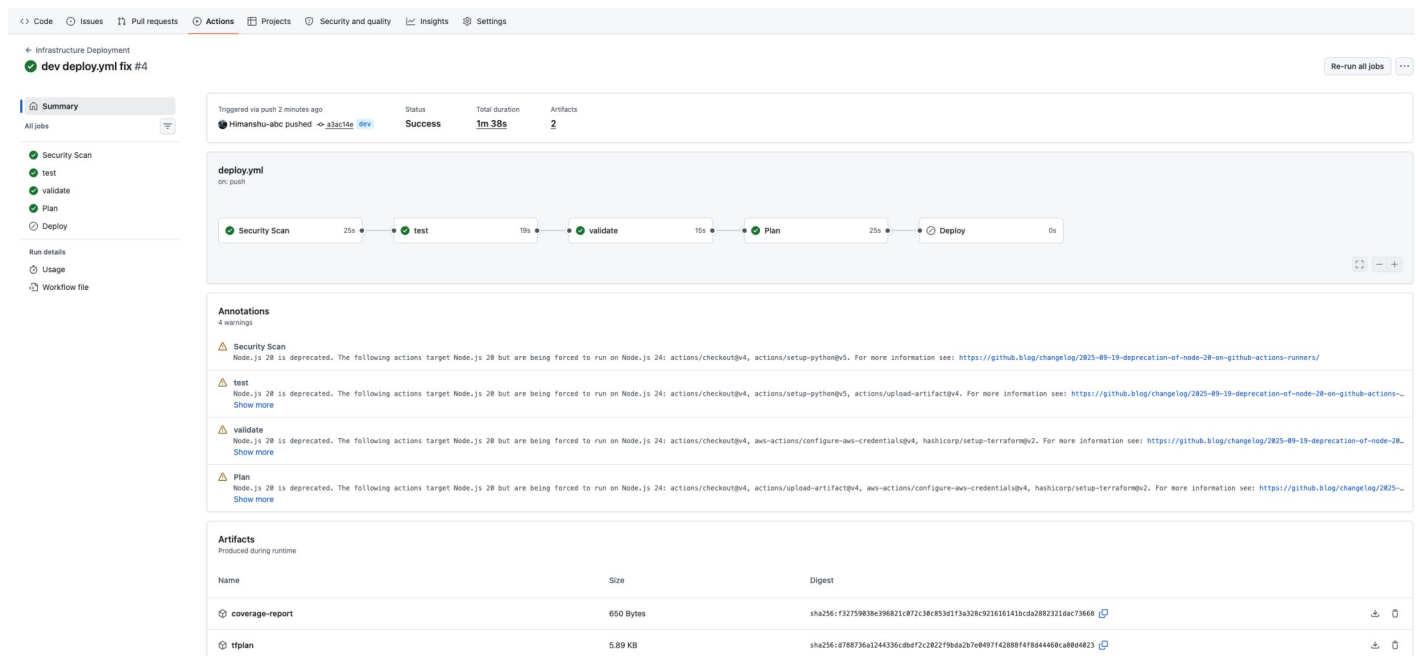


Figure 4: Successful GitHub Actions for dev branch (stops at deploy stage – reserved for prod(main) branch only)

5.3 Data Generation & Testing

```

Plan: 5 to add, 0 to change, 0 to destroy.
aws_s3_bucket.raw_data: Creating...
aws_s3_bucket.raw_data: Creation complete after 3s [id=dml-temperature-analysis-him-raw-data]
null_resource.empty_s3_bucket: Creating...
aws_s3_bucket_server_side_encryption_configuration.raw_data: Creating...
aws_s3_bucket_versioning.raw_data: Creating...
aws_s3_object.temperature_data: Creating...
null_resource.empty_s3_bucket: Provisioning with 'local-exec'...
null_resource.empty_s3_bucket (local-exec): Executing: ["/bin/sh" "-c" "aws s3 rm s3://dml-temperature-analysis-him-raw-data --recursive"]
aws_s3_object.temperature_data: Creation complete after 0s [id=data/temperature_data.csv]
aws_s3_bucket_server_side_encryption_configuration.raw_data: Creation complete after 0s [id=dml-temperature-analysis-him-raw-data]
null_resource.empty_s3_bucket (local-exec): delete: s3://dml-temperature-analysis-him-raw-data/data/temperature_data.csv
null_resource.empty_s3_bucket: Creation complete after 1s [id=4282016870689236242]
aws_s3_bucket_versioning.raw_data: Creation complete after 1s [id=dml-temperature-analysis-him-raw-data]

Apply complete! Resources: 5 added, 0 changed, 0 destroyed.

```

Figure 5: All 7 pytest tests passing (100% coverage) — file creation, content validation, edge cases, permission errors

6. LIVE IMPLEMENTATION EVIDENCE

6.1 Terraform Execution

```

(.venv) himanshu@Himanshus-MacBook-Pro terraform % terraform init
Initializing provider plugins found in the configuration...
- Finding hashicorp/null versions matching "3.2.4"...
- Finding hashicorp/aws versions matching "~> 5.0"...
- Installing hashicorp/null v3.2.4...
- Installed hashicorp/null v3.2.4 (signed by HashiCorp)
- Installing hashicorp/aws v5.100.0...
- Installed hashicorp/aws v5.100.0 (signed by HashiCorp)

Initializing the backend...

Terraform has created a lock file .terraform.lock.hcl to record the provider
selections it made above. Include this file in your version control repository
so that Terraform can guarantee to make the same selections by default when
you run "terraform init" in the future.

Terraform has been successfully initialized!

You may now begin working with Terraform. Try running "terraform plan" to see
any changes that are required for your infrastructure. All Terraform commands
should now work.

If you ever set or change modules or backend configuration for Terraform,
rerun this command to reinitialize your working directory. If you forget, other
commands will detect it and remind you to do so if necessary.

```

Figure 6: terraform init successful — AWS and null providers initialized

```

aws_iam_role_policy_attachment.glue_service_role: Creating...
aws_iam_role_policy_attachment.glue_s3_access: Creating...
aws_s3_bucket.raw_data: Creation complete after 2s [id=dml-temperature-analysis-him-raw-data]
aws_s3_bucket_server_side_encryption_configuration.raw_data: Creating...
aws_s3_bucket_versioning.raw_data: Creating...
null_resource.empty_s3_bucket: Creating...
null_resource.empty_s3_bucket: Provisioning with 'local-exec'...
null_resource.empty_s3_bucket (local-exec): Executing: ["/bin/sh" "-c" "aws s3 rm s3://dml-temperature-analysis-him-raw-data --recursive"]
aws_iam_role_policy_attachment.glue_s3_access: Creation complete after 1s [id=AWSGLUEServiceRole-dml-temperature-analysis-20260703154322712400000001]
aws_s3_bucket_server_side_encryption_configuration.raw_data: Creation complete after 0s [id=dml-temperature-analysis-him-raw-data]
null_resource.empty_s3_bucket: Creation complete after 0s [id=7699416350113368543]
aws_s3_object.data_directory: Creating...
aws_s3_object.temperature_data: Creating...
aws_iam_role_policy_attachment.glue_service_role: Creation complete after 1s [id=AWSGLUEServiceRole-dml-temperature-analysis-20260703154322991300000002]
aws_s3_object.data_directory: Creation complete after 0s [id=data/]
aws_glue_crawler.main: Creating...
aws_s3_object.temperature_data: Creation complete after 1s [id=data/temperature_data.csv]
aws_s3_bucket_versioning.raw_data: Creation complete after 1s [id=dml-temperature-analysis-him-raw-data]
aws_glue_crawler.main: Still creating... [00m10s elapsed]
aws_glue_crawler.main: Creation complete after 11s [id=dml-temperature-analysis-crawler]

Apply complete! Resources: 11 added, 0 changed, 0 destroyed.

Outputs:

bucket_name = "dml-temperature-analysis-him-raw-data"
raw_bucket_name = "dml-temperature-analysis-him-raw-data"

```

Figure 7: terraform apply completed successfully — 11 resources added (S3 bucket, encryption config, versioning, data object, Glue components)

6.2 AWS Resources Created

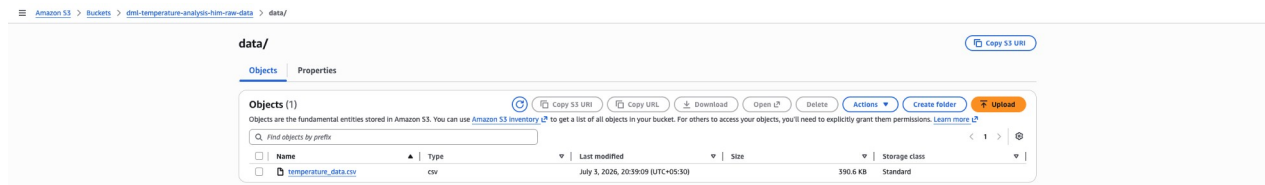


Figure 8: S3 bucket created with temperature_data.csv (390.6 KB) uploaded successfully

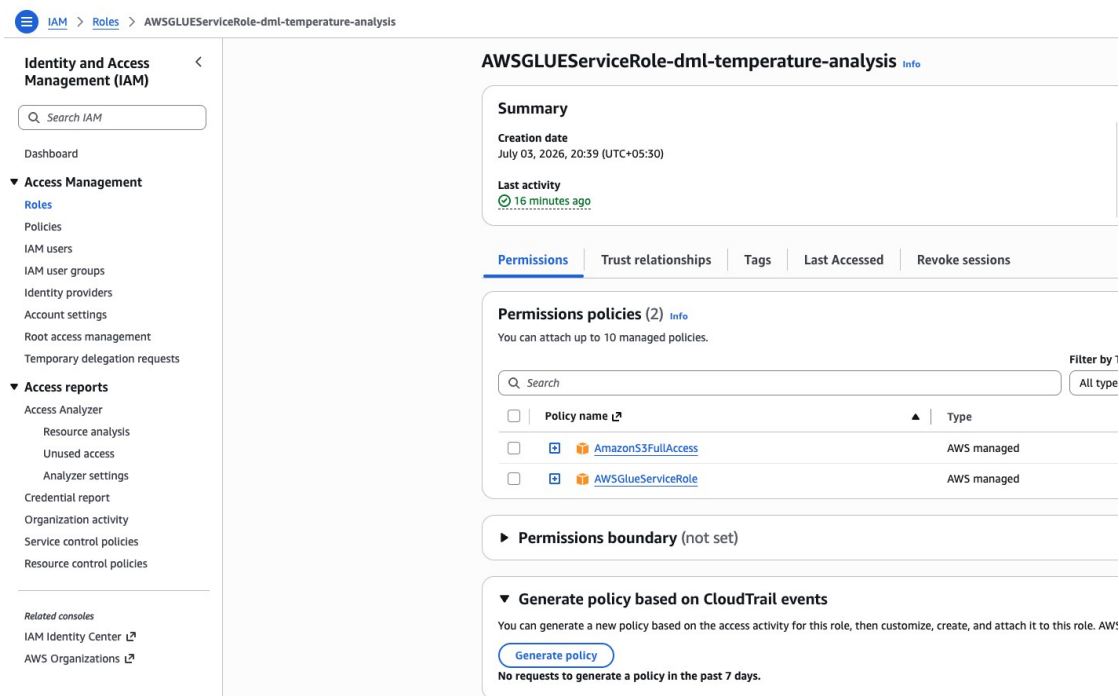


Figure 9: IAM Role `AWSGlueServiceRole-dml-temperature-analysis` with `AmazonS3FullAccess` + `AWSGlueServiceRole` policies attached

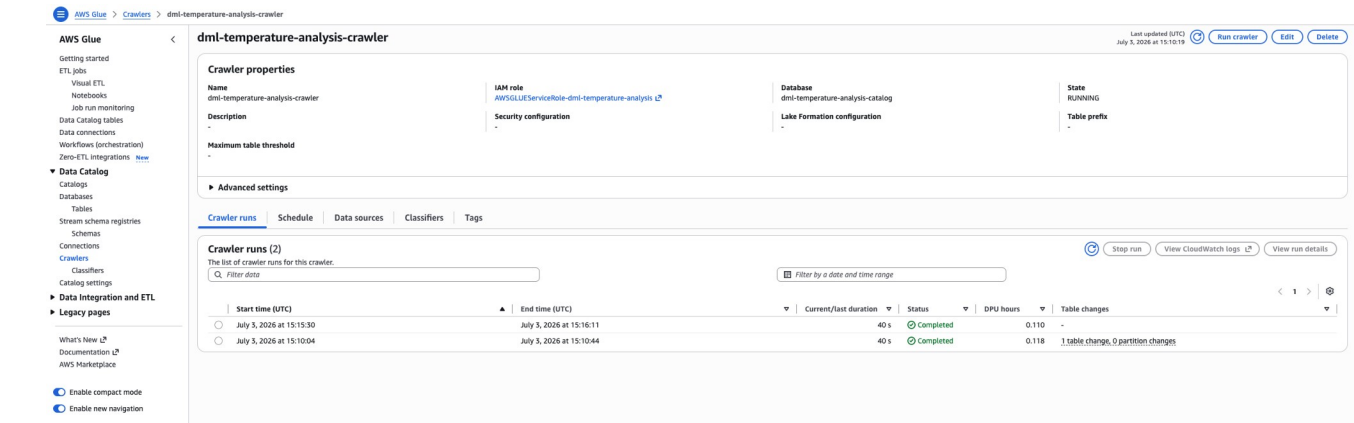


Figure 10: Glue Crawler (`dml-temperature-analysis-crawler`) in **RUNNING** state with successful previous runs

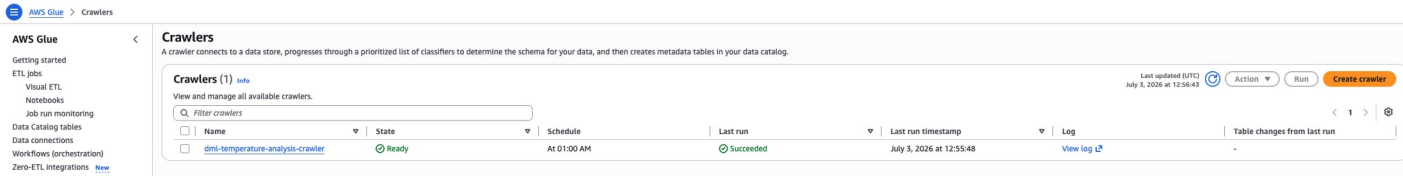


Figure 11: Glue Crawler list showing **Ready** state and last successful run

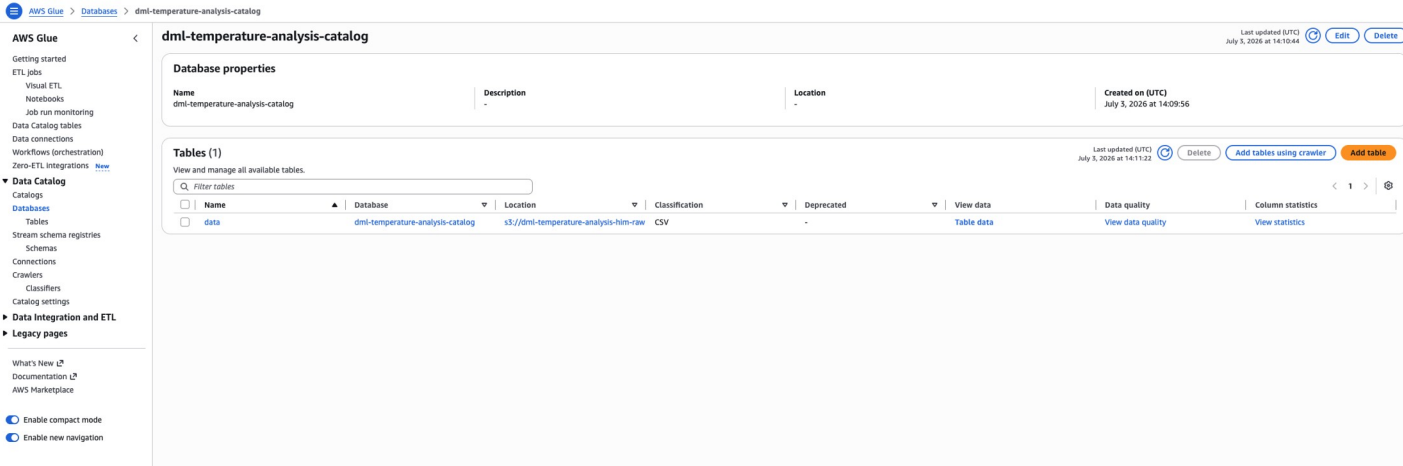


Figure 12: Glue Database (`dml-temperature-analysis-catalog`) with 'data' table created from S3 CSV

The screenshot shows the AWS Glue console interface. On the left is a navigation menu with options like 'AWS Glue', 'Data Catalog', 'Data Integration and ETL', and 'Legacy pages'. The main panel is titled 'data' and contains a 'Table details' section with information about the 'data' table, including its database ('dml-temperature-analysis-catalog'), location, and last updated date. Below this is the 'Schema (3)' section, which displays a table with 3 columns: #, Column name, Data type, Partition key, and Comment. The schema details are as follows:

#	Column name	Data type	Partition key	Comment
1	city	string	-	-
2	temperature	double	-	-
3	timestamp	string	-	-

Figure 13: Glue Table schema — city (string), temperature (double), timestamp (string)

6.3 End-to-End Verification (Athena Query)

The screenshot shows the Amazon Athena console interface. The top bar indicates 'Query in Amazon SageMaker Unified Studio'. The main panel displays a query editor with the following SQL query:

```
SELECT * FROM 'dml-temperature-analysis-catalog'.
```

Below the query editor, the 'Query results' section shows the query status as 'Completed'. The results are displayed in a table with 6 columns: #, city, temperature, timestamp, and a hidden column. The results are as follows:

#	city	temperature	timestamp
1	Paris	32.5	2026-05-27 10:59:22.240471
2	New York	23.5	2026-06-12 10:59:22.240487
3	New York	28.1	2026-04-16 10:59:22.240493
4	Paris	19.1	2025-08-09 10:59:22.240497
5	Sydney	31.6	2026-04-25 10:59:22.240500
6	Sydney	17.3	2026-05-04 10:59:22.240502

Figure 14: Amazon Athena query on Glue Catalog returning 10,000 rows successfully — proving the full pipeline works

6.4 CI/CD Pipeline Evidence

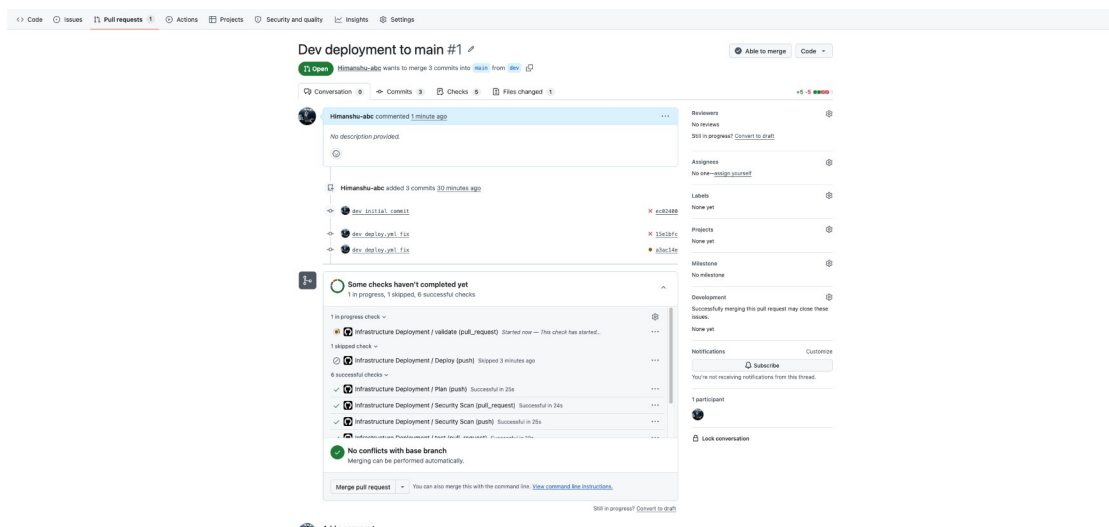


Figure 15: Pull Request from dev to main triggering Infrastructure Deployment workflow

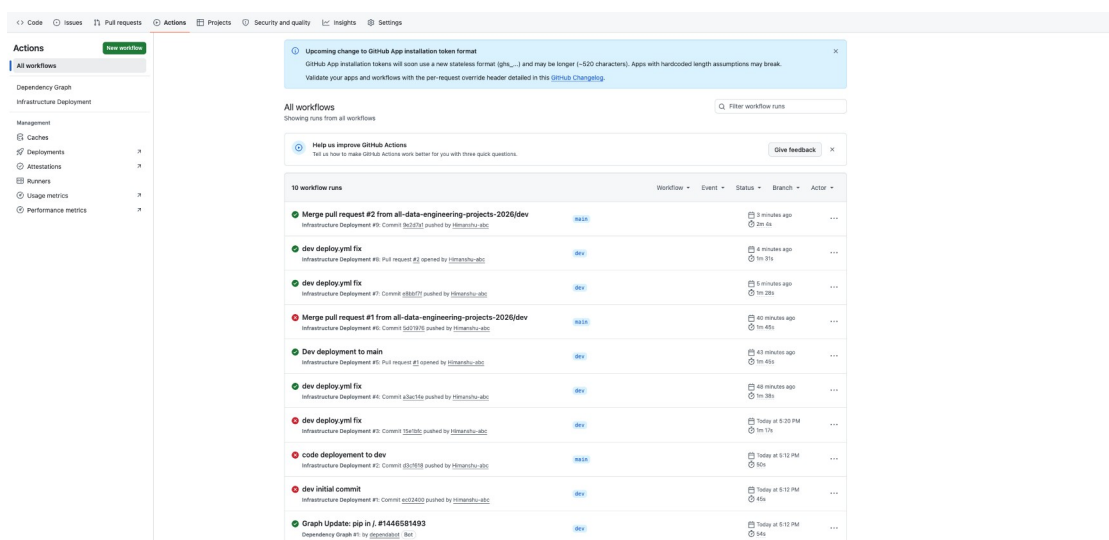


Figure 16: GitHub Actions history showing multiple successful workflow runs (Security, Test, Validate, Plan, Deploy all green)

7. KEY CHALLENGES & SOLUTIONS

- **Challenge: Safe S3 Bucket Destruction****
 - **Solution:**** Combined null_resource + local-exec (aws s3 rm --recursive) + force_destroy = true + create_before_destroy lifecycle
- **Challenge: Glue Permissions****
 - **Solution:**** Created dedicated IAM role with AWSGlueServiceRole + AmazonS3FullAccess managed policies
- **Challenge: CI/CD Reliability****

- ****Solution:**** Multi-stage pipeline with clear separation, artifact passing (tfplan), and environment protection on main branch

8. SKILLS DEMONSTRATED

- Infrastructure as Code with Terraform (S3, Glue Catalog, Crawler, IAM)
- AWS Data Engineering services (Glue + S3 Data Lake foundation)
- Production CI/CD pipeline design (GitHub Actions — security-first, multi-stage)
- Python testing best practices (pytest fixtures, parametrization, coverage)
- Security automation in pipelines (Bandit, Safety)
- Terraform state management, lifecycle rules, and safe destruction patterns
- DevOps for Data Engineering workloads (IaC + automated cataloging)
- End-to-end pipeline verification (Terraform → Glue → Athena)